

V8 Engine JIT & GC

L0 Essence: V8 引擎的本质是通过隐藏类（Map）将 JavaScript 的动态属性寻址转换为类似 C++ 的物理偏移寻址，并利用内联缓存（IC）和类型反馈向量收集运行期多态数据，驱动 Ignition 解释器与 TurboFan 编译优化及去优化（Deopt）控制流，配合 Cheney 双空间 Scavenger 与三色并发 Major MC 实现极致效率的内存周转。

[M1.1] V8 引擎管线（Execution Pipeline）与 Ignition/TurboFan 协作机制

V8 执行 JavaScript 代码时，首先通过 Parser 将源码解析为 AST，Ignition 解释器接收 AST 并编译生成小巧的字节码执行，同时负责收集运行时的类型反馈信息（Type Feedback Vector）。当函数执行频率达到阈值变热（Hot）时，TurboFan 优化编译器拉取字节码和反馈向量，结合 Speculation（推测）编译出极高效率的机器码；若运行期对象类型与假设冲突，则回退执行去优化（Deoptimization）重构栈帧落回 Ignition。

[M1.2] 极速非优化 JIT 编译器 Sparkplug 的定位与提速原理

Sparkplug 是介于 Ignition 解释器与 TurboFan 优化编译器之间的中层 JIT 编译器。它旨在快速生成机器码而不引入复杂的编译期分析或全局优化。Sparkplug 并不需要收集类型反馈，而是直接线性扫描 Ignition 字节码，并 1:1 翻译成对应的 CPU 机器指令，消除了字节码解码分发循环（Bytecode Dispatch）的解析开销，实现几乎零编译延迟的高效热启动提速。

[M1.3] 语法规析器（Parser）与延迟解析（Lazy Parsing）技术

为缩短首屏 JS 加载和编译卡顿，V8 引入了延迟解析机制。在初始加载时，只对处于顶层被立即调用的逻辑和函数执行“全解析”（Full Parse）生成 AST 并构建作用域链；对未执行的嵌套函数等执行“预解析”（Pre-Parse），只验证语法正确性并记录外层变量引用，跳过 AST 节点物理分配，直到函数被实际调用时才触发二次全解析。

[M1.4] 抽象语法树（AST）设计与作用域分析（Scope Analysis）

Parser 将 Token 流解析转换为层次分明的 AST 树。在此期间，Scope Analyzer 会对变量执行静态作用域分析，辨识每个变量的作用范围（如 Local、Context、Global）。当检测到内部嵌套函数通过闭包捕获了外部变量时，分析器会打破传统的栈分配限制，标记该外部局部变量将其“提升”分配至堆内存的 Context 对象中，确保生命周期安全。

[M2.1] 隐藏类（Map / Shapes）的核心设计与属性偏移量（Offsets）映射

JavaScript 的属性是动态且不规则的，V8 引入了 C++ 内部的“隐藏类”（名为 Map）进行静态化重用。Map 不直接包含属性值，而是保存属性名称与属性在对象存储槽（Object Slots）中的物理偏移量（Offset）的映射表。这样，V8 能够通过读取 Map 并在几条机器指令内直接依据偏移量读写属性值，避开了低效的动态哈希表查询。

[M2.2] 隐藏类转换链（Transition Chain / Trees）的生成与分支规则

JS 对象初始拥有一个空的默认 Map。当用户动态向对象添加属性时（例如先加 .x 再加 .y），V8 引擎并非反复为其分配独立 Map，而是为对象生成一条转换链：Map0（加 x）→ Map1（加 y）→ Map2。后续其他对象按照相同顺序添加属性时，会共享相同的 Map 转换树路径，从而极大减少 Map 分配开销并确保缓存重用。

[M2.3] 对象属性存储的三种物理形态：对象内属性（In-Object）、快属性（Fast）与慢属性（Slow/Dictionary）

V8 将对象的属性划分为三种物理形态：
1. 对象内属性（In-Object）: 属性值直接连续存储在 JSObject 物理对象头后面，寻址速度达到硬件极限（即单次指针加偏移）。
2. 快属性（Fast/Flat Properties）: 对象内属性槽存满后分配外部属性数组，由 Map 映射外部数组索引（依然是 $O(1)$ 寻址）。
3. 慢属性（Slow/Dictionary）: 当属性频繁删除或超出常值时，Map 结构退化为哈希字典存储，放弃隐藏类，属性查询效率回退至传统的字典遍历。

[M2.4] 隐藏类在数组（Elements）优化中的变体与基本元素类型（ElementsKinds）

V8 将对象的 Index 键值（数组元素）存储在独立的 Elements 数组中，并利用 Map 记录其特定的元素类型（ElementsKinds）。例如，存储连续整数时标记为 PACKED_SMI_ELEMENTS，带有未初始化空位时转换为 HOLEY_SMI_ELEMENTS。转换路径是单向且不可逆的（如 SMI → DOUBLE → ELEMENTS），尽可能保持在精细具体的 Kinds 上能产出效率极高的单指令 SIMD 机器码。

[M3.1] 类型反馈向量（Feedback Vector）结构与反馈插槽（Feedback Slots）

为了向 JIT 编译器提供类型支持，V8 为每个函数或闭包分配一个单独的 Feedback Vector。向量内部被划分为多个反馈插槽（Feedback Slots），每个插槽对应代码中的一个属性访问（Load/Store）或函数调用（Call Site）。插槽中初始化为空，随着代码的运行，不断填充和记录捕获到的对象 Map 和具体跳转目标地址。

[M3.2] 内联缓存（Inline Cache - IC）核心逻辑与状态转换图

内联缓存是一种通过在特定调用点缓存最近捕获的 Map 与属性偏移量以提高性能的技术。其状态转换包含以下过程：
• **UNINITIALIZED:** 初始状态，插槽为空。
• **MONOMORPHIC（单态）:** 调用点仅遭遇过 1 种 Map，缓存该 Map 和偏移量，后续查对 Map 成功即可直接读取（命中率 99% 的核心状态）。
• **POLYMORPHIC（多态）:** 遭遇过 2-4 种 Map，使用一个小型多态映射表（Map-Handler 对）轮流遍历。
• **MEGAMORPHIC（超态）:** 遭遇超过 4 种 Map，不再使用多态表，转而回退至全局 Stub Cache 或 C++ 运行时慢速字典查询。

L1 Logical Framework

- 管线协作与延迟解析:** 源码通过延迟解析按需构建 AST 并交由 Ignition 生成字节码，运行时动态生成隐藏类转换链（Transition Tree）实现属性快速寻址。
- 类型反馈与单态内联:** 在反馈插槽收集运行态 Map 并利用内联缓存（IC）消除慢路径查询，将单态访问点直接 JIT 编译为极高效率的直接寻址机器指令。
- 节点海优化与去优化机制:** TurboFan 将字节码展开为 Sea of Nodes 进行无锁拓扑全局优化，并利用猜想式编译技术在检查失败时通过去优化（Deopt）安全滑落回解释器。
- 新生拷贝与并发老生清扫:** 借助写屏障（Write Barrier）隔离跨代指针，新生代通过 Cheney 双空间无碎片紧凑拷贝，老生代通过并发标记、延迟清扫与 Compact 彻底清退死亡内存。

L2 Data Flow Topology

• JS Source → Parser (Full/Lazy) → AST → Ignition Bytecode → Interpreter Loop (Direct Threaded) → Access Property → Create/Transition Map (Hidden Class) → Fill Feedback Vector Slots → Monomorphic IC Status → Speculative JIT (TurboFan Sea of Nodes) → Optimized Machine Code → Type Mismatch (e.g. String passed to Num) → Eager Deoptimization → Reconstruct interpreter Frame → Bailout to Ignition → Allocations trigger GC → Scavenger (Cheney From To) / Major MC (Incremental Mark → Lazy Sweep Compact) → Write Barrier updates Remembered Set.

Trade-off Matrix: JIT & VM Execution

▮ 设计维度 (Design Dimension): 方案 A (Approach A) → 方案 B (Approach B) [折衷折度 (Trade-off Balance)]

▮ 执行架构 vs 编译体积: 基于栈式计算模型 (Stack-based, 如 clox) → 基于累加器寄存器模型 (Accumulator-based, 如 Ignition) [栈式计算模型指令结构极其简单，不用显式指定源与目的寄存器 ▮ 但累加器寄存器模型通过隐式累加器寄存器大幅减少指令的操作数个数，将字节码的整体物理大小压缩 30% 以上，显著节省内存下载和带宽开销 [牺牲了少许译码复杂度，换取了极致的字节码空间压缩率]]

▮ 属性寻址 vs 内存膨胀: 传统的动态哈希表查询 (Dynamic HashTable) → 隐藏类与偏移寻址 (Map & Offsets) [动态哈希表支持完全的动态自由属性增删且每个对象结构独立 ▮ 但 V8 的 Map 机制建立在共享 Transition Tree 基础之上，为每个对象绑定只读偏移映射，使属性寻址只需单条指针寻址；代价是产生 Transition 频繁分化及 Map 对象的常驻开销 [以少量 Map 内存膨胀换取十几倍的寻址效率提升]]

▮ 优化决策 vs 编译停顿: 单级解释执行与 TurboFan 直接优化 → 引入 Sparkplug 中层非优化编译器的多级 JIT [无中层编译器时，函数冷热状态迁移简单且无过渡编译开销 ▮ 但 Sparkplug 的引入通过 1:1 翻译字节码，以接近零编译停顿代价清除了 Ignition 解码分发开销，避免了直接升腾至 TurboFan 的长等待 [引入额外的 Sparkplug 代码与多级栈帧过渡，换取更平滑的冷热提速曲线]]

▮ 动态假定 vs 安全滑落: 静态类型声明强制执行 (Static Casts) → JIT 猜想式编译与 Eager/Lazy 去优化 (Bailout) [静态类型从底层保证了执行效率和安全性且无需任何运行时回退机制 ▮ 但为了保持 JS 的绝对动态语义，V8 利用 Speculation 猜想编译快速执行；在 Map 检查校验失败时利用极其复杂的栈帧重构执行 Bailout 滑落，产生短暂的 CPU 去优化停顿 [开发了极复杂的 deoptimizer，以换取极速执行和动态特性共存]]

[M3.4] 属性写入中的内联缓存（Store IC）与原型链寻址优化

Store IC 不仅要在当前 Map 层面加速属性写入，还必须监控原型链的健康。V8 原型链对象的 Map 改变会使下属所有 IC 失效。为了加速原型访问，V8 在原型对象的 Map 中注册 Prototype User 引用，当原型链发生任何写入或结构修改时，触发 Invalidate 机制使全链相关 IC 回退到 UNINITIALIZED 重新加热。

[M4.1] Ignition 解释器字节码设计与累加器寄存器（Accumulator）模型

Ignition 采用基于寄存器与累加器混合的架构模型。大多数字节码指令默认将隐式的累加器寄存器（Accumulator, acc）作为其源操作数或目标操作数（例如 LdaSmi [10] 代表加载常量 10 至 acc; Add r1 代表将 r1 与 acc 相加结果存入 acc）。这相比双/三操作数指令，极大减小了生成的字节码物理体积，降低了内存占用与读取开销。

[M4.2] 解释器分发表（Bytecode Dispatch Table）与直接线程化解释（Direct Threaded Code）

Ignition 的指令译码并不使用传统的循环 switch(opcode)。它在初始化时为所有字节码 Handler 分配一段连续的机器码地址表（Dispatch Table）。在每个字节码 Handler 执行的尾部，会内联调用直接线程化解释代码；直接获取下一条字节码索引，并计算物理 Handler 地址，利用尾调用（Tail Call）或汇编级 jmp 瞬间跳转，消除了循环体自身的判定和栈帧开销。

[M4.3] 解释器局部环境与栈帧（Stack Frame）的物理布局

当 Ignition 执行 JS 函数时，在系统物理栈上建立栈帧。V8 栈帧的头部存有指向当前 JSFunction 实例的指针、外部 Context 上下文、以及指向字节码指令数组的指针。栈帧中紧接着分配当前函数所需的局部变量和临时计算结果存储寄存器，通过相对栈指针的负偏移量实现快速的相对寄存器访问。

[M4.4] 类型回饋收集字节码：‘LdaNamedProperty’ 与 ‘KeyedStoreIC’

V8 设计了能够感知回饋向量插槽的专用读写字节码。如 LdaNamedProperty r0, [0], [2]，第一参数 r0 为宿主对象，[0] 是常量池内属性名 .x 的索引，[2] 是插槽编号。解释器执行该 Handler 时会读取 r0 的 Map 并检索属性，然后主动把该 Map 与寻址路径更新到 Feedback Vector 的 2 号插槽内，供后期 TurboFan 优化。

[M5.1] Sea of Nodes 编译期中介表示（Intermediate Representation - IR）设计

TurboFan 采用高级的 Sea of Nodes 编译中介表示。该设计统一了控制流（CFG）与数据流（DFG）的表示方式。所有数据值、内存效果（Effect）与跳转逻辑（Control）在图中均为平等的 Node 节点，它们之间仅通过数据依赖边和控制依赖边关联，使得编译器优化时可以并发且无锁地执行图修剪、重构和优化 Passes。

[M5.2] 冗余 Map 检查消除优化（Map Check Elimination）与逃逸分析（Escape Analysis）

在 Sea of Nodes 优化阶段，TurboFan 利用两项关键优化提升运行速度：

1. Map Check Elimination (Map 检查消除): 分析并沿控制流方向推导对象 Map 的不变区间。一旦确认某对象在此段内必然只含有特定 Map，便彻底擦除后续冗余的检查机器指令。

2. Escape Analysis (逃逸分析): 检测生命周期被局限在当前函数内的对象。一旦对象未逃逸，直接消除其在堆上的物理内存分配，将其成员打散为栈上的独立局部变量标量（Scalar Replacement）。

[M5.3] JIT 编译器的去优化（Deoptimization / Bailout）物理机制

JIT 的编译假设是非常激进的。一旦运行期假定失效（如 TurboFan 认为参数必定是 Int，但用户传入了 String，导致 Map Check 指针比较失败），机器码中内联的陷阱指令会跳转至 Deoptimizer 代码。它会暂停执行，拉取 CPU 机器寄存器的当前值，物理重构成 Ignition 对应的栈帧结构，并将控制权交还给字节码解释器（Bailout），保证程序的绝对安全和语义一致性。

[M5.4] 软去优化（Soft Deopt）与硬去优化（Lazy/Eager Deopt）的触发差异

- **Eager Deopt (硬同步去优化):** 执行过程中类型检查立刻失败，必须原位、瞬间重构栈帧落回解释器。
- **Lazy Deopt (硬异步去优化):** 函数机器码正在跑，但外部原型对象或者全局结构发生了更改（例如覆盖了 Array.prototype.push），导致之前编译出来的优化假设失效，在遇到代码中预埋的检查标记（SafePoint）时触发去优化。
- **Soft Deopt (软去优化):** 仅仅是推测遇到了非常用分支（如数组越界访问），触发编译器将该分支的代码退回重新编译，不涉及严重的当前帧栈重构。

[M6.1] V8 堆内存分代布局（V8 Heap Layout）与空间划分

V8 物理堆空间分为新生代（New Space）与老生代（Old Space）。新生代专门管理生命周期短的活动对象。老生代则接收收新生代晋升（Promote）的常驻对象。此外，还设有了大对象空间（Large Object Space，直接接纳大容量对象避免拷贝开销）和代码空间（Code Space，独立保护并运行 JIT 生成的机器指令）。

[M6.2] 物理页（Page）组织与写屏障（Write Barrier）辅助机制

堆内存被切分成 1MB 的物理页（Page）。垃圾回收通过写屏障（Write Barrier）协助跨代引用管理：当老生代对象的属性被写入指向新生代对象的引用时，写屏障会拦截该写操作，并将目标老生代对象的所在物理页地址，注册到卡表（Remembered Set / Card Table）中，确保新生代 GC 扫描时免去全堆遍历。

[M6.3] 新生代 Scavenger 垃圾回收算法与 Semi-space 双空间模型

新生代（New Space）划分成大小相等的 To-space 与 From-space，发生 Scavenge 回收时，GC 线程线性扫描 From-space 中的活跃对象，并将其按原样紧凑拷贝到 To-space 中（如果对象存活过多次，则直接拷贝搬迁到 Old Space 老生代）。拷贝完毕后，From/To 空间角色对调，From-space 物理重置归零，消除内存碎片并实现 O(1) 极速周转。

[M6.4] 写屏障（Write Barrier）在 Scavenger GC 中的物理执行路径

Scavenger 回收时，GC 的可达性分析根集（Roots）除了 CPU 计算栈、全局上下文之外，还直接吸纳了老生代 Remembered Set 卡表中注册的老生代 Page 物理槽位。GC 只扫描这些卡页中包含的新生代引用，无须扫描没有注册在卡表里的海量老生代对象，从而使新生代 GC 停顿通常控制在 5 毫秒以内。

[M6.5] 老生代 Major MC 垃圾回收：Mark-Sweep-Compact 算法流程

Major MC 专职处理老生代回收，执行以下三步流程：

1. **Mark (三色标记):** 并发多线程追踪全局引用网，通过灰色工作链黑化活跃对象。
2. **Sweep (物理清扫):** 将判定为白色的死亡对象地址回收并挂载到 Free List（空闲链表），为新分配腾出槽位。
3. **Compact (整理压实):** 将碎片化的存活对象向物理页连续地址移动对齐，重构对象内部的所有物理指针，彻底解决长期运行导致的堆内存碎片化问题。

[M6.6] 并发标记（Concurrent Marking）与写障碍（Write Barrier）的标记协作

为了消除 GC 停顿，三色标记通过后台并发线程进行（Concurrent Marking），与 JS 执行线程并行。在此期间，如果 JS 线程修改了已扫描对象的引用（例如把黑色对象指向白色对象），写屏障会强制拦截并将目标白色对象标记染为灰色，推入 GC 工作队列中，防止活引用被错误漏扫回收。

[M6.7] 增量标记（Incremental Marking）与延迟清扫（Lazy Sweeping）的降停顿原理

- **Incremental Marking (增量标记):** 将巨大的标记时间切成小的时间片，穿插在 JS 宏任务循环中，主线程只停顿微小间隔。
- **Lazy Sweeping (延迟清扫):** 标记结束后并不集中清扫。清扫工作在分配（Allocation）需要时才按物理页分步按需进行，从而完全分散了 Sweeping 时长，避免了长时间主线程卡顿。

[M6.8] 垃圾回收的并行并发优化: Parallel vs Concurrent vs Incremental 区别

V8 将三种垃圾回收技术深度结合以极致降低停顿：

- **Parallel (并行):** 触发 GC 时主线程挂起（STW），但多条辅助 GC 线程并行参与，按核心数缩短停顿时间。
- **Concurrent (并发):** 主线程在继续跑 JavaScript 代码，GC 线程完全在后台静默执行标记/清扫。
- **Incremental (增量):** 将回收阶段分解为小片交替执行。

Zone T1: V8 Bytecodes & Parameters

LdaSmi [constant] : 加载一个小整数（Smi）常值至累加器（acc）中。

\\LdaNamedProperty r0, [name_idx], [slot_idx] : 读取寄存器 r0 对象的指定属性，并向反馈插槽记录 Map 类型回饋。\\StaNamedProperty r0, [name_idx], [slot_idx] : 向寄存器 r0 对象写入指定属性，并通过内联缓存执行快速写入。\\Ldar r1/Star r1 : Load accumulator from register r1 / Store accumulator to register r1. \\CallProperty1 r1, r2, r3, [slot_idx] : 对 r1 对象的方法进行调用，传递 1 个参数，并记录方法分发类型回饋。

Zone T2: JIT Deopt & GC Diagnostics

DeoptimizeReason 机器码退役原因: \\kWrongMap : 输入对象的隐藏类 Map 与优化机器码编译期预设不一致，触发 Eager Deopt. \\kOutOfBounds : 数组索引访问超出当前 Elements 数组界限，退出优化回退至解释器以安全扩容。\\kNotASymbol / kNotANumber : 参数类型假设错误。\\ElementsKind 元素类型优化状态: \\PACKED_SMI_ELEMENTS : 密集 Smis 数组，效率最高的数组类型。\\HOLEY_DOUBLE_ELEMENTS : 含有孔洞（Holes）的双精度浮点数数组，访问时需处理 Hole 默认返回值。\\DICTIONARY_ELEMENTS : 数组退化为哈希字典，物理寻址速度最慢。\\GC_REMAINING_ERROR_BUDGET (GC 触发扩展): \\v8_flags.gc_global_amount : 全局垃圾回收堆字节容量分配上限阈值参数。\\v8_flags.incremental_marking : 决定是否开启增量标记的运行时控制开关。